Comparative Analysis Report

PRAHARI

PRAHARI Concept vs. Securade HUB (hub-main)

Executive Summary

This report presents a technical and conceptual comparison between the existing **Securade.ai HUB (hub-main)** project and the newly proposed **PRAHARI AI Vision Safety & Risk Intelligence** architecture.

While *hub-main* operates as a reactive, edge-based computer vision tool primarily capturing basic PPE compliance via 2D bounding boxes, *PRAHARI* introduces a proactive, spatial risk intelligence paradigm. PRAHARI expands the scope beyond basic compliance to include comprehensive workforce resource monitoring, spatial metric awareness (e.g., precise edge distance tracking, dynamic load evaluation on scaffolds), multi-layered dashboards, and deep ecosystem integrations (LMS, domain models).

1. Core Paradigm & Philosophy

Securade HUB (hub-main)

Paradigm: Reactive Compliance

- Acts as a standalone edge node connecting to RTSP streams.
- Detects discrete, point-in-time violations (e.g., missing helmet).
- Notifies via Telegram bots and saves static JPEGs.

PRAHARI Architecture

Paradigm: Proactive Risk & Resource Intelligence

- Analyzes risk dynamically with multi-camera spatial tracking.
- Predicts near-misses (e.g., dwell time near an unprotected slab edge).

- Simple dashboards summarizing daily occurrence counts.

- Consolidates safety analytics with site productivity/resource monitoring (people, scaffolds, MEWPs).
- Replays incidents with context and provides actionable EHS recommendations.

2. Technical Capabilities Matrix

Capability Domain	Securade HUB (hub-main)	PRAHARI (Proposed)
Computer Vision Space	2D Pixel Coordinates Bounding-box overlap logic in flat pixel space.	Calibrated Spatial Zones Real-world metrics (e.g., 0.8m distance), depth context.
Event Tracking & Context	Frame-by-Frame Evaluates individual frames; triggers on single instances.	Temporal Dwell Tracking Evaluates duration (e.g., 6 seconds) and multi-camera ID tracking.
Safety Check Depth	Basic PPE & Zones Hardhats, vests, masks, 2D proximity to vehicles.	6 Comprehensive Families Includes Scaffold Integrity, Working at Heights, Line of Fire.
Risk Evaluation Engine	Binary Logic Triggered (1) or Not Triggered (0).	4-State Nuanced Rules PASS, WARNING, FAIL, UNCLEAR with severity tiering.
Evidence Capture	Static Image (JPEG) Stores single frame snapshot to disk.	Rich Event Replay 12-sec video evidence clipping with timeline, distance, and PPE metadata.

Capability Domain	Securade HUB (hub-main)	PRAHARI (Proposed)
Dashboards & Analytics	Basic Bar Charts Simple daily violations and aggregate formulas.	Multi-faceted BI Risk spatial heatmaps (24h), Pareto charts, total workforce visibility.

3. System Architecture Evolution

Current Architecture (hub-main)

Built as a monolithic Python orchestration layer (`securade.py`) integrating YOLOv7 inference (OpenVINO/CUDA) and Streamlit/Dash local web apps. It relies on a simplistic plugin system (`plugins/base_plugin.py`) and local JSON configs. Does not integrate with broader enterprise safety ecosystems.

Target Architecture (PRAHARI)

Designed as an enterprise-grade, layered platform:

- **Vision Engine (Tvashta Core):** Offloads raw object detection, spatial mapping, and track-lets to a dedicated vision microservice.
- **Intelligence Layer (PRAHARI):** Houses the domain-specific business logic—the 6 Safety Check Families.
- **Risk Engine:** Processes data streams for rule execution and anomaly detection.
- **Integrations Layer:** Hooks directly into Vasudhava (LMS for training), Laya (Data domains), and Vishva3D (Digital Twins).

4. 30-Day MVP Execution Plan (Transition to PRAHARI)

Sprint Phase	Objective	Hub-Main Baseline Utilization
Days 1-5: Setup & Reqs	Define hazard zones, establish baseline metrics, set up pilot area.	Leverage existing IP camera integration and RTSP ingestion tools.
Days 6-10: Build Core	Deploy Tvashta Vision Core; camera calibration and spatial tracking.	Refactor current YOLOv7 pipeline to support temporal tracking (Deepsort/ByteTrack) and spatial grid mapping.
Days 11-15: Safety Intel	Implement the 6 Core PRAHARI Check Families.	Upgrade `safety_app.py` logic from basic polygon overlaps to dynamic distance calculations.
Days 16-20: Resource Intel	Track workforce density, scaffolding, MEWPs, and logistics.	Add new model classes and aggregate reporting modules.
Days 21-25: Proactive Analytics	Enable risk heatmaps, near-miss pattern recognition.	Replace basic Dash app with advanced BI frameworks (e.g., React/Next.js or advanced Plotly).
Days 26-30: Pilot & Validation	Staged scenarios, performance feedback, and LMS/EHS integration.	Integrate APIs with Vasudhava LMS for closed-loop training.

5. Conclusion & Recommendations

The **hub-main** repository serves as an effective foundation for edge-based computer vision ingestion and basic model inference. However, realizing the **PRAHARI** vision requires a substantial architectural shift.

To bridge the gap, the development team must upgrade the CV pipeline from 2D frame-level analysis to temporal, spatial 3D-aware tracking. Furthermore, the dashboard and alerting

systems must evolve from simple static snapshots to comprehensive, interactive risk intelligence interfaces complete with video replay and ecosystem API integrations.

Generated by Antigravity AI • Confidential System Analysis • 2026